



UD3

INTRODUCCIÓN A JAVASCRIPT

3.5 MANEJO DEL DOM

MP_0373

Lenguaje de marcas y
Sistemas de gestión de
la información

Introducción

Un documento HTML no deja de ser un conjunto de caracteres que por sí mismos no podrían mostrar ningún sitio web. Es el navegador al interpretar dicho documento quien realmente muestra un sitio. Para ello crea el DOM y una vez que el navegador lo ha creado, es posible alterarlo mediante JavaScript, alterando así, la visualización de la página.

Modelo de Objetos del Documento (DOM)

El DOM es una representación en memoria del documento HTML que muestra un navegador. Cada uno de los elementos HTML que conforman el documento, se convierte en objetos en memoria con sus propiedades y métodos a los que es posible acceder mediante una API. Todo cambio que se realice en el DOM, cambiará la visualización del sitio web en el navegador.

```
<html>
  <body>
    <h1 id="saludo">Hola Mundo</h1>
  </body>
</html>

const elementoSaludo = document.getElementById('saludo');
elementoSaludo.innerHTML = "Adiós mundo";
// Tras ejecutar esto, en el navegador se visualizará el título h1 mostrando:
Adiós mundo
```

En el anterior ejemplo, JavaScript está modificando la visualización de la página utilizando para ello el DOM y su API. Al ejecutar JavaScript en un navegador está disponible la variable global `document` la cual hace referencia al documento. Al documento, por ejemplo, se le puede pedir mediante el **método** `getElementById()`, que busque un elemento con un id determinado.

El elemento con id *saludo* se guarda en la constante `elementoSaludo`. Gracias a la propiedad **innerHTML** de un elemento HTML para escribir dentro de él un nuevo contenido: `_Adiós mundo_`. Una vez realizado este cambio, el navegador mostrará en pantalla la visualización del nuevo DOM, es decir, mostrará un título h1 con el contenido *Adiós mundo* (reemplazando al anterior texto).

Eventos

Los eventos son sucesos que le ocurren a elementos HTML, JavaScript se puede suscribir a dichos eventos y actuar en consecuencia. Por ejemplo, que el usuario pulse sobre un botón dispara el evento `onClick` y JavaScript al suscribirse a ese evento puede actuar en consecuencia mostrando una alerta.

Un evento por tanto es una manera de comunicación: un elemento dispara un evento y hay **manejadores** que lo tratan. Un manejador es una función que se ejecuta en el momento en el que se lanza un evento al que está suscrito.

La manera recomendada de utilizar eventos en JavaScript es a través del método `addEventListener`:

```
<button id="button"></button>
const button = document.getElementById('button');
button.addEventListener('click', function(event) {
  console.log("Botón pulsado!");
});
```

En el anterior ejemplo, el método `addEventListener` está escuchando el evento `click`. El elemento `button` al ser un elemento HTML podrá disparar este evento en el momento en el que el usuario hace `click`. Cuando esto ocurra, se ejecutará el manejador asociado al evento `click` que es la función que muestra por consola el mensaje «Botón pulsado!».

<https://developer.mozilla.org/es/docs/Web/Events> esta es la enorme lista de eventos DOM que pueden disparar elementos. Estos son algunos de los más utilizados:

- ⊕ Eventos del ratón: `mousedown`, `mouseenter`, `mouseleave`, `mousemove`, `mouseout`, `mouseover`, `mouseup`, `dblclick`, `click`.
- ⊕ Eventos táctiles: `touchstart`, `touchmove`, `touchend`, `touchcancel`.
- ⊕ Eventos del teclado: `keydown`, `keypress`, `keyup`.
- ⊕ Eventos de formulario: `focus`, `change`, `submit`.
- ⊕ Eventos de la ventana: `scroll`, `resize`, `load`.

Document ready

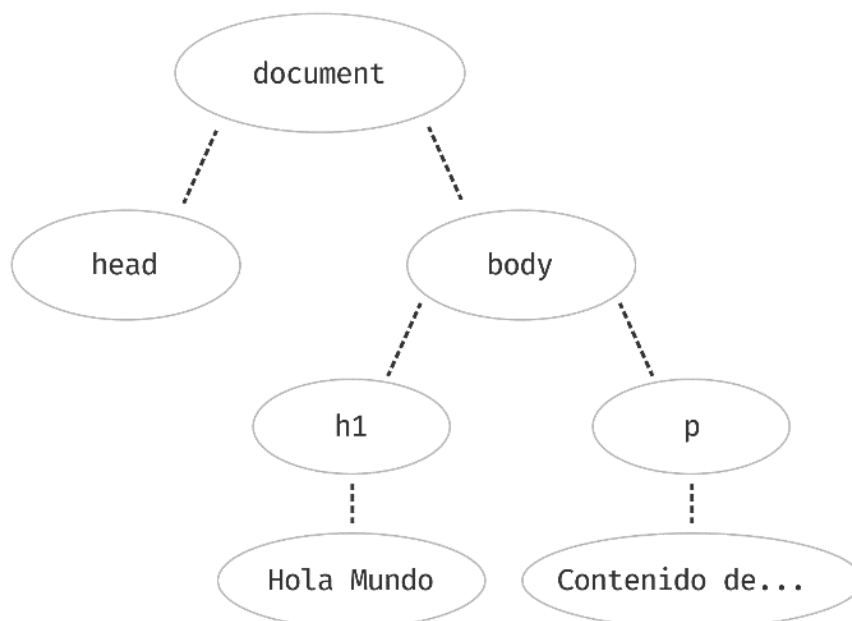
La creación del DOM no es un proceso inmediato: el navegador para mostrar una página tiene que leer el documento HTML y convertirlo a un objeto en memoria. Antes de tener el DOM listo, el navegador empezará a ejecutar JavaScript, es por ello por lo que **el código JavaScript que vaya a interactuar con el DOM tiene que esperar a que el DOM se haya generado**, de otra manera no podrá interactuar con él porque literalmente, no hay nada con lo que interactuar.

Para ello lo más sencillo es añadir un manejador al evento DOMContentLoaded que dispara el documento cuando el DOM ya está listo:

```
document.addEventListener('DOMContentLoaded', function(event) {  
    // Código JavaScript que manipula el DOM  
})
```

Selectores y métodos de acceso

Un documento HTML no es más que un árbol de nodos: Hay un nodo padre document del que cuelgan el resto de los elementos HTML, estos elementos HTML son **nodos** del árbol y si un elemento HTML tiene texto dentro, es un nodo de tipo texto.



```
<html>
  <head>
</head>
  <body>
    <h1>Hola Mundo</h1>
    <p>Contenido de un párrafo</p>
  </body>
</html>
```

Conocer esta estructura de árbol es importante para recorrer el DOM en busca de elementos: cada nodo tendrá únicamente un padre (excepto document) y un padre podrá tener uno o varios hijos. Tanto desde document como desde cualquier nodo se puede recorrer el DOM. Los métodos que proporciona la API para seleccionar elementos son los siguientes:

- ⊕ getElementById(id): Devuelve el elemento con un id específico.
- ⊕ getElementsByName(name): Devuelve los elementos con un valor del atributo name específico.
- ⊕ getElementsByTagName(tagname): Devuelve los elementos con un nombre de tag específico.
- ⊕ getElementsByClassName(classname): Devuelve los elementos con un nombre de clase específico.
- ⊕ querySelector(selector): Devuelve un único elemento que corresponda con el selector. El formato del selector es el mismo que el de un selector CSS.
- ⊕ querySelectorAll(selector): Devuelve un array con los elementos que correspondan con el selector. El formato del selector es el mismo que el de un selector CSS.

```
<p class="parrafo" id="parrafo1">
  <span class="estilado">
    Nueva
  </span>
  Camiseta blanca XL
  <span class="precio">
    9.3€
  </span>
</p>
<p class="parrafo" id="parrafo2">
```

```

<span class="estilado">
  Envío gratis
</span>
Camiseta negra XL
<span class="precio">
  9.3€
</span>
</p>

```

```

document.getElementsByClassName('precio'); // Devuelve los dos precios
document.getElementById('parrafo1'); // Devuelve el primer párrafo
document.querySelector('.estilado'); // Devuelve el primer elemento span con
contenido "Nueva"
document.querySelectorAll('.estilado'); // Devuelve los dos elementos con
etiqueta span con contenido "Nueva" y "Envío gratis"
document.getElementById('parrafo3'); // Devuelve null

```

Manipulación del DOM

Accediendo al DOM se puede manipular su contenido, el navegador de primeras mostrará lo que le llegue desde el documento HTML (primera carga del DOM), pero mediante JavaScript se puede modificar ese DOM creado y la visualización en el navegador reflejará esos cambios.

Manipulando el documento

- ⊕ `createElement(etiqueta)`: Crea un elemento HTML, como atributo se puede seleccionar la etiqueta que tendrá ese elemento.
- ⊕ `appendChild(node)`: Permite añadir un hijo a un elemento.
- ⊕ `removeChild(child)`: Elimina el nodo hijo que se indica con child.

Manipulando elementos

Manipular clases:

- ⊕ `classList.add('clase')`: Añade la clase que se pasa por parámetro.
- ⊕ `classList.remove('clase')`: Quita la clase que se pasa por parámetro.
- ⊕ `classList.toggle('clase')`: Si tiene la clase que se le pasa por parámetro la quita y si no la tiene, la pone.

Manipular contenido:

- ⊕ `innerHTML`. Mediante esta propiedad se puede añadir contenido HTML a cualquier elemento.
- ⊕ `innerText`. Utilizando esta propiedad se puede añadir texto plano a un elemento.

Manipular cualquier atributo:

- ⊕ `getAttribute(nombre)`: Método que obtiene el valor del atributo cuyo nombre se pasa por argumento.
- ⊕ `setAttribute(nombre)`: Método que escribe el valor del atributo para el nombre que se pasa por argumento.

En el siguiente extracto de código se realizan operaciones habituales de manipulación del DOM como crear elementos, añadir una clase y atributo:

```
// Crea un párrafo
const paragraph = document.createElement('p');

// Se le añade contenido
paragraph.innerText = 'soy un párrafo';

// Se le añade una clase
paragraph.classList.add('tipo1');

// Se añade al body
document.appendChild(paragraph);

// Crea una imagen
const image = document.createElement('img');

// Se añade el atributo src con la ruta de la imagen
image.setAttribute('src', 'image.jpg');

// Se añade al body
document.appendChild(image);
```

El anterior código JavaScript, genera el siguiente resultado en el DOM:

```
<body>
  <p class="tipo1">soy un párrafo</p>
  
</body>
```

Intervalos y timeout

setTimeout() y setInterval() son dos métodos incorporados de JavaScript que permiten ejecutar algo pasado un determinado tiempo.

setTimeout() ejecutará la función que se le pase por argumento pasado un tiempo determinado, mientras que setInterval() ejecutará la función cada un determinado tiempo de manera infinita. clearTimeout() y clearInterval() permiten borrar estos temporizadores.

```
// Recibe dos argumentos: la función a ejecutar y el tiempo (en milisegundos)
que va a esperar a ejecutarla
```

```
const saludar = function() {
  console.log("Hola");
}
```

```
setTimeout(saludar, 1000);
```

```
// Pasado 1 segundo mostrará por consola: Hola
```

```
setInterval(function() {
  console.log("hola");
}, 1000);
```

```
// Cada segundo mostrará por consola: Hola
```

```
// Tanto setTimeout y setInterval al crearse devuelven un id
```

```
// Ese id es necesario para poder "limpiar" el intervalo
```

```
const intervalId = setInterval(function() {
  console.log("hola");
}, 1000);
```

```
setTimeout(function() {  
    clearInterval(intervalId);  
}, 3000);  
  
// Segundo 1: hola  
// Segundo 2: hola  
// Segundo 3: no se ve nada por consola porque se ha limpiado el intervalo
```